I. Use Case Description	
Use Case Name	Reasoning About Action Feasibility from Physical Properties
Use Case Identifier	UC-ActionFeasibility-01
Source	LACAI Lab, Department of Computer Science, Rensselaer Polytechnic Institute
Point of Contact	Hyejin Park, Department of Computer Science, Rensselaer Polytechnic Institute, parkh12@rpi.edu Xiye Fan, Department of Cognitive Science, Rensselaer Polytechnic Institute, fanx4@rpi.edu Liangliang Zhang, Department of Computer Science, Rensselaer Polytechnic Institute Zhangl41@rpi.edu Alex Clark, Department of Electrical, Computer & Systems Engineering, Rensselaer Polytechnic Institute, clarka14@rpi.edu
Creation / Revision Date	September 18, 2026 (created); revised October 23, 2026 (Assignment 8),October 29, 2026 (Assignment 9), November 4, 2026 (Assignment 10), and November 13, 2026 (Assignment 11)
Associated Documents	OE_11_TeamAffordance.rdf; OE_11_TeamAffordance_Individuals.rdf; OE_11_TeamAffordance_Queries.txt; OE_11_TeamAffordance_CuratedTermList; OE_11_TeamAffordance_ConceptualModel; OE_11_TeamAffordance_Report

II. Use Case Summary	
Goal	Develop an extensible ontology that supports machine-interpretable and explainable reasoning about physical action feasibility. The ontology represents how physical features of objects and object parts, together with relevant structural relationships and target conditions, relate to the physical requirements of actions. This representation is intended to support affordance reasoning in situations where object category or conventional function alone is not sufficient to determine what actions are physically possible. The current implementation focuses on the physical action CUT in everyday kitchen contexts.
Requirements	The ontology should represent physical entities, object parts, physical features and feature values, actions, action requirements, assessment contexts, and the semantic relationships that connect them. It should support representations in which a candidate, target, and action are evaluated together, including cases where target conditions affect the requirements applied to a candidate. Feasibility judgments should be grounded in explicitly represented physical information, including candidate and target features, relevant object parts and part relationships, and their relationships to action requirements. Object category or conventional function must not by itself determine feasibility; individual names serve only as identifiers. Physical features are assumed to be available as structured information. The ontology is not responsible for object recognition or for estimating physical features from images or object names. The reasoning system should also distinguish explicit incompatibility from insufficient information and support explanations that identify the physical evidence relevant to a judgment. The ontology should support automated inference with a description logic reasoner and rules, including classifying requirement mappings by their conditions, inferring which action requirements an assessment satisfies or conflicts with, assigning a final feasibility judgment according to the shared evaluation rule in Section IV, and classifying candidates by their part structure.

Scope	CUT is treated as a physical action that can be supported by different combinations of physical features, relevant object parts, part relationships, and target conditions, rather than as an action associated exclusively with a particular object category or cutting method. The ontology represents candidate and target physical features, object-part structure, action requirements, and requirement mappings that describe how relevant physical conditions relate to CUT feasibility. The current implementation is situated in everyday kitchen contexts and assumes a reference performer with the hand-scale force and grip of an ordinary adult, whether the action is performed by a person or by a household robot. The current implementation is situated in everyday kitchen contexts and assumes a reference performer with human-scale manipulation capabilities. This assumption provides a fixed context for interpreting the qualitative CUT requirements; individual actor capabilities, such as robot-specific force limits, grasping, and motion control, are not explicitly modeled. Physical features are treated as structured information available to the reasoning system rather than information that must be estimated by the ontology itself. Object recognition, visual feature estimation, safety, convenience, and social appropriateness are outside the intended reasoning task. The modeling approach is designed so that it can later be extended to additional actions, physical features, and interaction contexts.
Priority	High — representing action feasibility from physical properties is central to the intended reasoning task and provides the foundation for later extensions of the ontology.
Stakeholders	Primary stakeholders are developers of household and assistive robots and of language-model-based task planners that must decide which available objects can be used to perform an action. Secondary stakeholders are researchers in knowledge representation, affordance reasoning, and embodied AI who study how physical features and object structure determine action feasibility.
Description	The ontology is motivated by the need for physically grounded affordance reasoning in embodied AI systems, such as household robots and autonomous agents. These systems must determine whether available objects can physically support intended actions, including situations where conventional tools are unavailable or unfamiliar objects must be considered as substitutes. **Why it is being developed.** Large language models can support task planning through learned semantic associations between objects and their conventional functions (e.g., "knife → cutting"). However, these associations alone do not establish whether a particular object can perform an action on a given target. This creates a gap between language-based task planning and physically grounded affordance reasoning [7], [8], [12]. **What problem it solves.** Our ontology addresses this gap by explicitly representing the physical features of candidate objects, their parts and structural relationships, target conditions, and action requirements. Rather than relying on object names or conventional functions, the reasoning system evaluates whether represented physical properties satisfy the

	requirements for CUT and provides explanations grounded in those properties and relationships. **Future use.** The ontology is intended as a foundational knowledge component for interpretable physical feasibility reasoning in embodied AI systems. Potential applications include robot task planning, evaluating affordance predictions made by language models, and studying unconventional tool use. The current implementation focuses on controlled CUT scenarios in kitchen environments, rather than comprehensive robotic manipulation or physical simulation.
Actors / Interfaces	**Primary Actor – Robot Task Planner (automated agent):** The planning component of a household or assistive robot, potentially supported by a large language model. Before selecting or substituting a tool, it submits structured descriptions of candidate objects and targets to the ontology-based reasoning system. The returned judgment and explanation can inform subsequent planning decisions, while physical execution and manipulation control remain outside the scope of the ontology. **Primary Actor – Robotics Developer:** A developer who maintains a robot's tool knowledge. The developer adds or updates object descriptions (physical features, parts, and part relationships), requests their assessment and structural classification, and uses the results to configure the robot's behavior. **System – Ontology-Based Reasoning System:** Uses the ontology, a description logic reasoner, and rules to retrieve the requirements of CUT, select the requirement mappings that apply to the target, compare candidate feature values with the required levels, classify candidates by part structure, and return judgments with explanations. **Secondary Actor / Interface – Knowledge Base / Data Sources:** Supplies the structured feature values, part structure, part relationships, and target conditions used to populate the ontology, such as the output of a robot's perception module or curated annotations.
Pre-conditions	The ontology and its associated knowledge base have been loaded, and the query and reasoning functions are available. The knowledge base contains the represented requirements for CUT, relevant requirement mappings, and the structured candidate and target information needed for the assessment. Physical features and part information are assumed to be supplied or already represented rather than estimated by the ontology.
Post-conditions	The system returns the appropriate response to the query. The query does not add, remove, or modify information in the underlying knowledge base, and the system remains available for subsequent queries.
Triggers	The use case is triggered when (1) a robot task planner must confirm that a selected or substitute object can cut a target before executing a step; (2) a robot task planner must choose a substitute from the available objects because the usual tool is missing; or (3) a robotics

	developer adds or updates a tool description and requests its assessment and structural classification.
Performance Requirements	No specific runtime performance requirement is imposed. Priority is given to correctness, traceability, and explainability of ontology-based reasoning rather than response speed.
Assumptions	Relevant candidate and target physical features, object-part information, and part relationships are assumed to be available as structured information. The ontology does not recognize objects or estimate their physical properties from images, lexical names, or conventional functions. The current representation focuses on qualitative physical features and relationships rather than comprehensive quantitative physical simulation.
Open Issues	(1) Missing information: the rules assign Insufficient Information only when a required feature is explicitly recorded as unspecified (hasUnspecifiedFeatureType); a general treatment of missing values without such a marker remains open under open-world semantics. In Assignment 11 the marker yields an explicit inferred hasUnresolvedRequirement, but it must still be asserted for each missing feature. (2) Conflict coverage: conflicts are inferred for the two modeled CUT requirements and the high-hardness target only; other requirement and level combinations need further rules. (3) Part-level features: requirement satisfaction is inferred through hasPart (rule S9), but conflicts are not yet inferred from the features of parts. (4) Coverage: requirement mappings currently cover high-hardness targets and two CUT requirements; further targets, requirements, and actions remain to be added. (5) Judgment policy: the current rules do not fully enforce the intended priority between infeasible, feasible, and insufficient-information outcomes, so some data combinations could produce incompatible judgments. Resolving competing judgments and handling newly supplied information remain open for team review; no rules were changed during CQ3 validation. (6) Scalability and qualitative feature levels: The current ontology uses qualitative feature levels (Low, Medium, and High) within controlled CUT scenarios, without universally defined numerical thresholds. These levels support the current reasoning tests but may limit scalability when integrating larger datasets or more diverse objects. Future extensions could provide more explicit operational definitions and evidence-based numerical mappings for selected physical features. Such quantitative extensions are outside the scope of the current course implementation.

III. Usage Scenarios

Usage Scenario Framing

The following scenarios illustrate how an embodied AI system or its developer could use property-grounded affordance reasoning to evaluate physical action feasibility. The scenarios focus on situations where object identity and conventional function alone are insufficient for determining whether CUT can be performed.

Each scenario highlights a different reasoning capability: evaluating a candidate against target-dependent requirements (Scenario 1), identifying suitable candidates from available alternatives (Scenario 2), and comparing objects with different part structures (Scenario 3).

The competency questions in Section VII provide controlled, verifiable test cases derived from these scenarios. They are intended to evaluate the ontology's reasoning capabilities rather than demonstrate a fully implemented robotic system.

Scenario 1 — Verifying a Substitute Tool Before Acting

A household assistance robot is helping an older adult prepare vegetable soup. The next recipe step is to cut a carrot into pieces, but the kitchen knife is in the dishwasher. The robot's language-model-based task planner proposes using a rigid plastic card from the counter instead. Because this suggestion comes from general language knowledge rather than from the physical properties of the objects actually present, the robot must verify it before acting; an unverified substitute could fail, damage the object, or waste time. The robot's perception module has already recorded the card's edge thickness and rigidity and the carrot's hardness. The planner sends these descriptions to the ontology-based reasoning system and asks whether the card can cut the carrot. The system determines the CUT requirements that apply to a hard target, compares them with the card's features, and returns a judgment with an explanation. The planner uses the returned judgment and explanation to determine whether the proposed substitute should be considered for the task. Actual physical execution would require additional manipulation and safety checks beyond the ontology's feasibility assessment.

Modeling implication: requirements must depend on the features of the target (RequirementMapping), and feature levels must be explicit so that a candidate's features can be compared with the required levels.

Scenario 2 — Choosing a Substitute from Available Objects

In a shared office kitchen, a service robot is asked to prepare a snack plate with sliced carrots, but no knife is available. Its perception module lists the objects within reach (a rigid plastic card, a wooden rolling pin, and a kitchen sponge) together with their recorded physical features. Instead of choosing by object name, the task planner asks the reasoning system which of these objects can satisfy the CUT requirements for the carrot. The system evaluates every candidate against the same target-dependent requirements and returns three groups: candidates that satisfy all requirements, candidates whose represented features conflict with a requirement, and candidates whose information is incomplete. The planner selects a qualifying candidate or, if none qualify, tells the user that the step cannot be completed with the available objects and explains why. Modeling implication: each candidate needs its own Assessment with the same target and action, and the model must distinguish a conflicting feature from a missing one.

Scenario 3 — Comparing Alternative Part Structures for the Same Action

A robotics developer is extending a kitchen robot's tool library. The library contains a knife, described by a single blade part. The developer adds a pair of kitchen scissors, described by two blade parts that interact when the handles close. Before deploying the new tool, the developer needs to know whether the scissors meet the same CUT requirements as the knife for the same target, and how the ontology distinguishes the two structures, because these tools have different part structures that may require

different manipulation strategies when integrated into a robotic system. The developer submits both tool descriptions; the system evaluates the blade features against the CUT requirements, classifies each tool by its part structure, and returns a comparison with explanations.

Modeling implication: parts and their relationships (ObjectPart, hasPart, isPartOf, interactsWith) and structure classes (MultiPartObject, InteractingPartObject) are needed to represent and compare different physical realizations of the same action.

IV. Basic Flow of Events

Shared evaluation rule

The following describes the intended evaluation policy shared by the scenarios; the current rules implement the tested configurations with the limitations noted below. This is the single feasibility rule used throughout this use case and implemented in the ontology rules. For each assessment (one candidate, one target, and the CUT action), the system selects the requirement mappings whose target condition matches the represented features of the target; each mapping specifies the level required for one CUT requirement. A requirement is satisfied when the candidate, or one of its parts, has a represented value of the concerned feature at the required level (rules S1 and S9); it is in conflict when the candidate has a represented value of that feature at a different level (rules S5 and S6); and it is unresolved when the value of the concerned feature is explicitly recorded as unspecified (hasUnspecifiedFeatureType). The judgment is Feasible when every applicable requirement is satisfied (rules S3, S10, and S4); Infeasible when at least one requirement is in conflict (rule S7); and Insufficient Information when no requirement is in conflict but a required feature is unresolved (rule S8). Missing information is never treated as a conflict. Assignment 11: an unresolved requirement is now represented explicitly, because the reasoner infers hasUnresolvedRequirement for the assessment, and the SWRL rules carry descriptive names: S1 = InferSatisfiedRequirementFromMappedFeatures, S3 = InferTwoCUTRequirementsSatisfiedFromFeatures, S4 = AssignFeasibleJudgment, S5 = DetectRigidityConflict, S6 = DetectEdgeThicknessConflict, S7 = AssignInfeasibleJudgment, S9 = InferSatisfiedRequirementFromPartFeatures, S10 = ClassifyTwoCUTRequirementsSatisfied. The former S8 is replaced by MarkUnspecifiedFeatureRequirementAsUnresolved (infers hasUnresolvedRequirement) and by InsufficientWhenEdgeUnresolved, InsufficientWhenRigidityUnresolved and InsufficientWhenBothUnresolved (assign Insufficient Information).

The current rules implement this intended policy for the tested configurations, but do not enforce its outcome priorities for all possible data combinations. See Open Issues for limitations concerning conflict coverage, competing judgments, and updates when additional information becomes available.

Basic Flow Narrative — Scenario 1: Verifying a Substitute Tool Before Acting
The robot task planner submits the card and carrot descriptions and asks whether the card can cut the carrot. The system retrieves the CUT requirements and the mappings that apply to a high-hardness target, compares the edge

thickness and rigidity of the card with the required levels, derives a judgment using the shared evaluation rule, and returns it with an explanation. The planner then executes or rejects the step.

Basic / Normal Flow of Events			
Step	**Actor (Requester)**	**Actor (System)**	**Description**
1	Robot Task Planner		Submits the card and carrot descriptions and asks whether the card can cut the carrot.
2		Ontology-Based Reasoning System	Retrieves the requirements of the CUT action (rigidity and edge thickness).
3		Ontology-Based Reasoning System	Retrieves the represented features of the carrot and selects the mappings whose target condition matches (hardness = high).
4		Ontology-Based Reasoning System	Determines the required levels (rigidity = high, edge thickness = low) and retrieves the feature values of the card.
5		Ontology-Based Reasoning System	Records each requirement as satisfied, in conflict, or unresolved; if any requirement is unresolved, follows the alternate flow.
6		Ontology-Based Reasoning System	Derives the judgment and generates an explanation listing the features, mappings, and requirements involved.
7	Robot Task Planner		Receives the judgment and explanation and executes or rejects the cutting step.

Basic Flow Narrative — Scenario 2: Choosing a Substitute from Available Objects

The robot task planner submits the descriptions of all available objects (card, rolling pin, and sponge) and of the carrot, and asks which objects can cut the carrot. The system creates one assessment per candidate with the same target and action, applies the shared evaluation rule to each, and returns the qualifying, conflicting, and unresolved candidates, each with an explanation.

Step	Actor (Requester)	Actor (System)	Description
1	Robot Task Planner		Submits the candidate and target descriptions and asks which candidates can cut the carrot.
2		Ontology-Based Reasoning System	Retrieves the requirements of the CUT action (rigidity and edge thickness).
3		Ontology-Based Reasoning System	Determines the required levels from the hardness of the carrot once for all candidates and creates one assessment per candidate, linked to the same target and action.
4		Ontology-Based Reasoning System	Evaluates the feature values of each candidate against the required levels.
5		Ontology-Based Reasoning System	Groups each candidate as qualifying, in conflict, or unresolved (alternate flow) and continues until all candidates are evaluated.
6		Ontology-Based Reasoning System	Returns the groups with an explanation for each candidate.
7	Robot Task Planner		Selects a qualifying candidate or reports that no available object can perform the step.

Basic Flow Narrative — Scenario 3: Comparing Alternative Part Structures for the Same Action

The robotics developer submits the knife (one blade part) and the scissors (two interacting blade parts) and asks whether each can meet the CUT requirements for the same target and how their structures differ. The system evaluates the blade features against the requirements, classifies each tool by its part structure, and returns both results with a comparison.

Step	Actor (Requester)	Actor (System)	Description
1	Robotics Developer		Submits the knife and scissors descriptions, including parts, part features, and part interactions, with the same target.
2		Ontology-Based Reasoning System	Retrieves the requirements of the CUT action (rigidity and edge thickness).
3		Ontology-Based Reasoning System	Determines the required levels for the target and retrieves the parts of each tool and the feature values of those parts.
4		Ontology-Based Reasoning System	Evaluates the blade features of each tool against the required levels.
5		Ontology-Based Reasoning System	Classifies each tool by part structure (number of distinct parts and whether the parts interact).

6		Ontology-Based Reasoning System	Compares the requirement results and structural classifications of the two tools.
7		Ontology-Based Reasoning System	Returns a judgment, structural classification, and explanation for each tool.
8	Robotics Developer		Receives the results and records the appropriate manipulation strategy for each tool.

V. Alternate Flow of Events

Alternate Flow Narrative — Insufficient Feature or Part Information
This alternate flow may occur during any of the three Basic Flows when the reasoning system determines that one or more pieces of information required to evaluate an applicable CUT requirement are not available in the supplied candidate description or represented knowledge. The missing information may concern a relevant physical property, part structure, or relationship between parts. Missing information is not treated as evidence of infeasibility. The three scenarios are not handled in sequence; they all use the same single step. The system marks the affected result (the queried candidate, one candidate in a set, or one tool) as insufficient information rather than infeasible, finishes evaluating anything else that has enough information, and returns the results to the requesting actor with the missing information identified.If sufficient additional information is later provided, the system resumes evaluation.

Alternate Flow of Events			
Step	**Actor (Requester)**	**Actor (System)**	**Description**
A1		Ontology-Based Reasoning System	Detects that information required to evaluate an applicable CUT requirement is missing from the supplied candidate description or represented knowledge..
A2		Ontology-Based Reasoning System	Identifies the missing physical property, part structure, or part relationship and does not treat its absence as evidence of infeasibility. .
A3		Ontology-Based Reasoning System	Marks the affected result (queried candidate, one candidate in a set, or one tool) as insufficient information, finishes evaluating any remaining candidates or tools with enough information, and returns the results to the requesting actor with the missing information identified.
A4	Requesting actor		May provide additional physical-property or part information, select another candidate, or end the query..

A5		Ontology-Based Reasoning System	If sufficient information becomes available, resumes evaluation at the appropriate step of the corresponding Basic Flow.

VI. Use Case and Activity Diagram(s)

The following diagrams illustrate the actors, system components, and reasoning activities involved in the three usage scenarios. The use case diagram summarizes the primary interactions with the ontology-based reasoning system, while the architecture diagram shows where the ontology is used within the overall system.Three activity diagrams then show the detailed flows for (1) feasibility verification and explanation, (2) choosing a substitute from available objects, and (3) comparison of alternative part structures and property relationships.

Diagram 1 — High-Level Architecture / Overview

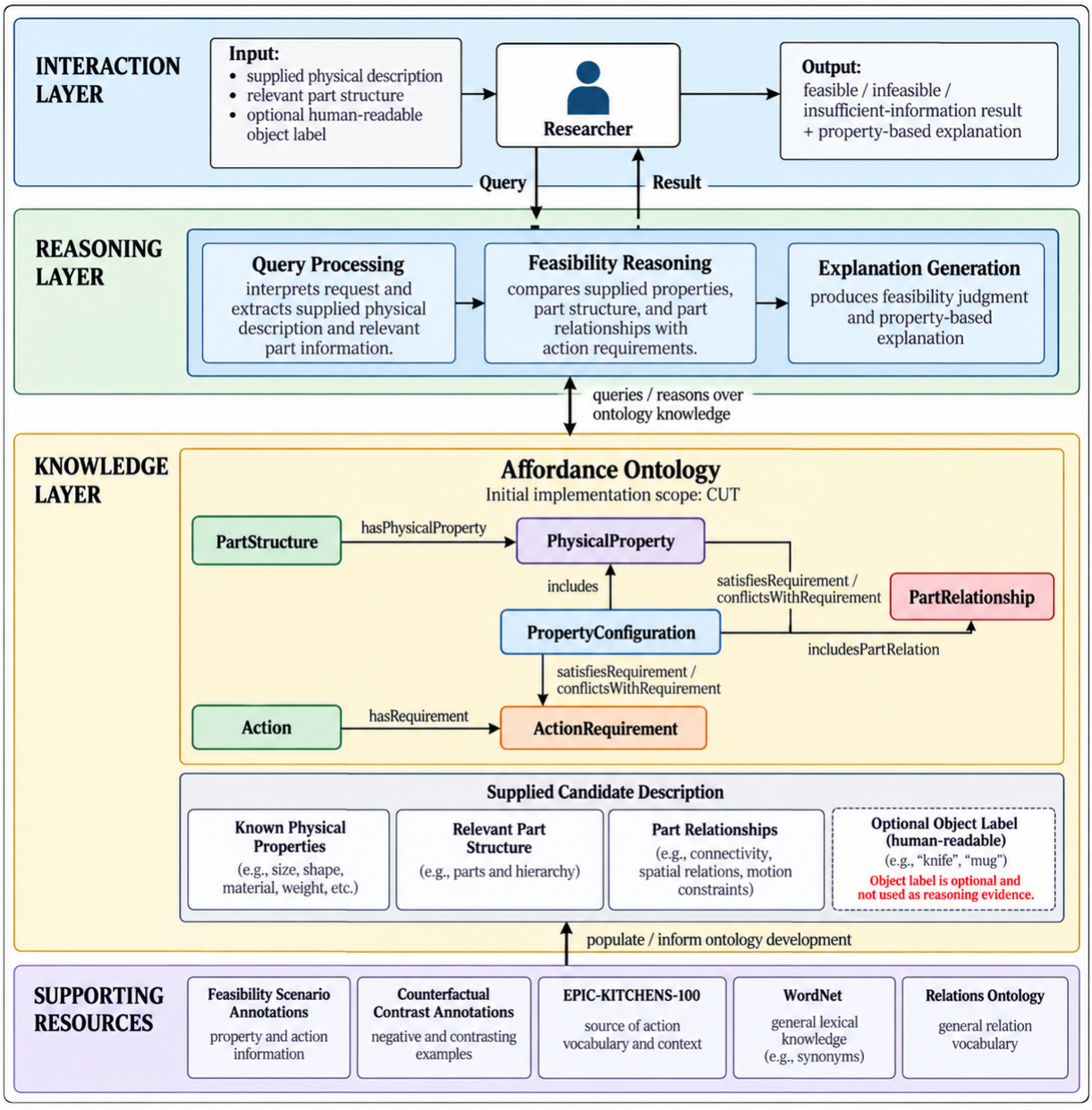


Diagram 2 — Use Case Diagram

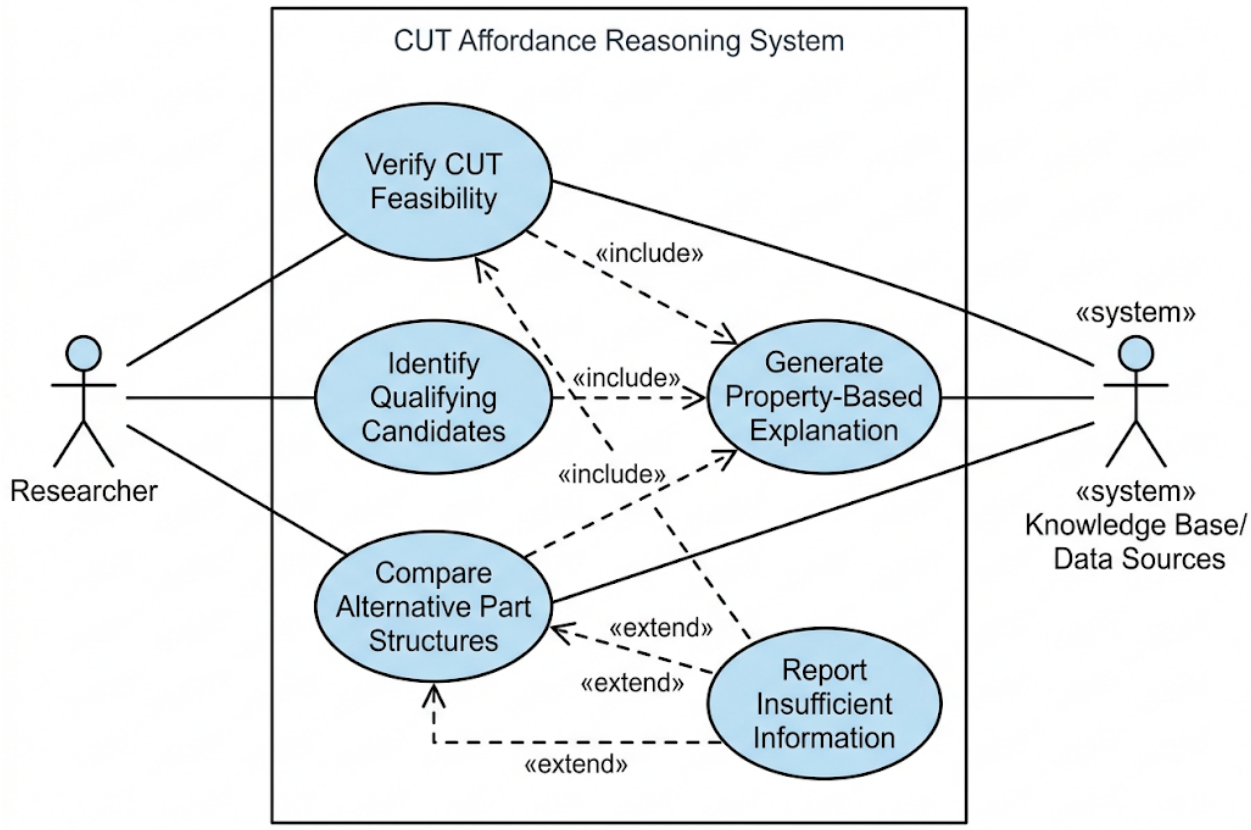
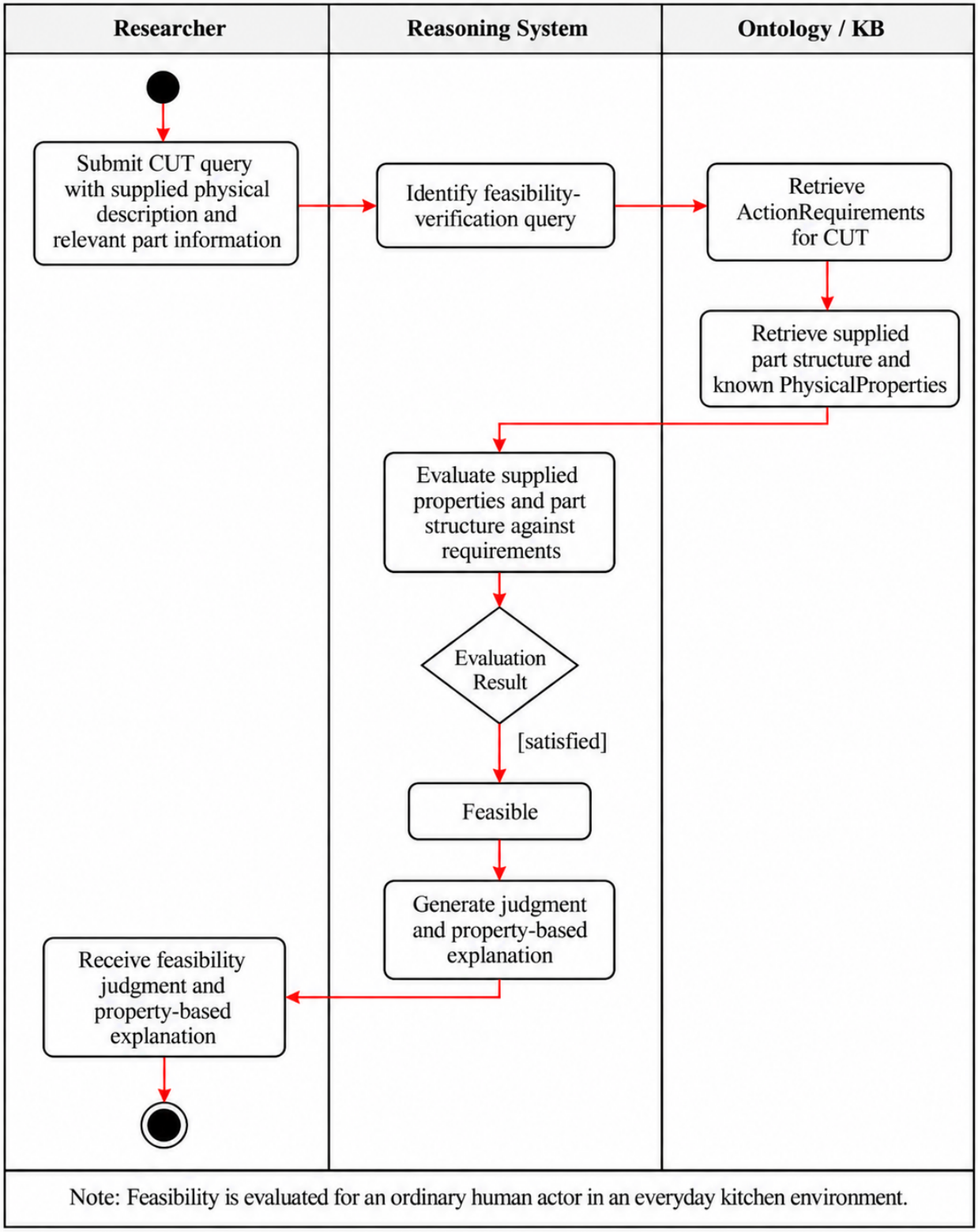


Diagram 3 — Activity Diagram: Scenario 1

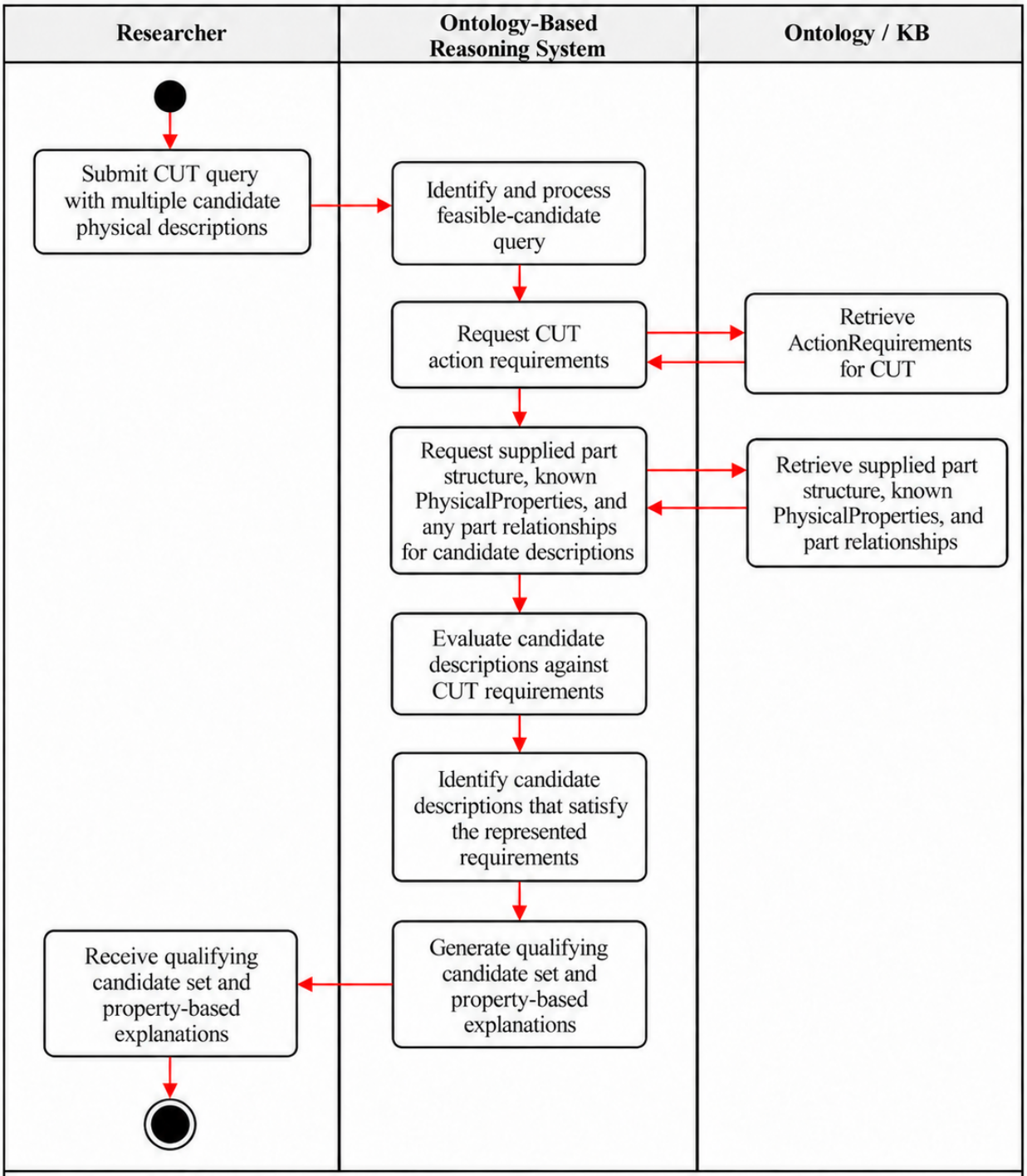
Scenario 1 applies the shared evaluation rule in Section IV: the candidate is feasible when all applicable requirements are satisfied, infeasible when at least one requirement is in conflict, and insufficient information when no requirement is in conflict but a required feature is unresolved. The merge joins these outcomes before the explanation is returned. This flow supports CQ1 and CQ6.



Note: Feasibility is evaluated for an ordinary human actor in an everyday kitchen environment.

Diagram 4 — Activity Diagram: Scenario 2

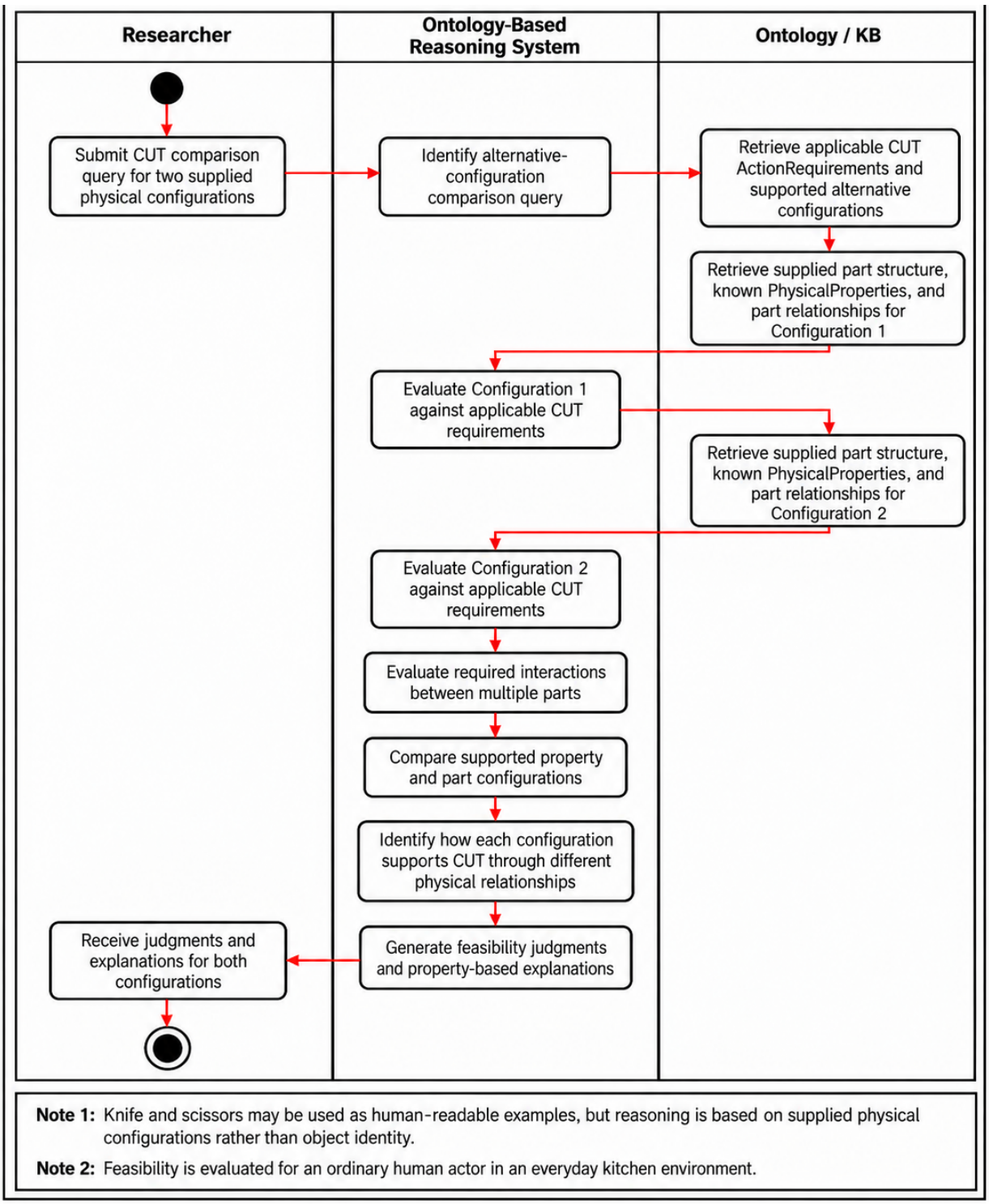
Scenario 2 evaluates every supplied candidate and records its judgment before continuing the loop. An unresolved candidate does not stop the search and is not placed in the infeasible group. The shared evaluation behavior is the system-side requirement evaluation and classification shown in Diagram 3. This flow supports CQ2, CQ4, and CQ5.



Note 1: Feasibility is evaluated for an ordinary human actor in an everyday kitchen environment.

Diagram 5 — Activity Diagram: Scenario 3

Scenario 3 applies the same evaluation to each tool independently, classifies each tool by its part structure, and compares the results, preserving an insufficient-information result for either tool. The comparison does not assume that both tools are feasible. This flow supports CQ3.



Note 1: Knife and scissors may be used as human-readable examples, but reasoning is based on supplied physical configurations rather than object identity.

Note 2: Feasibility is evaluated for an ordinary human actor in an everyday kitchen environment.

VII. Competency Questions

This section lists six competency questions. CQ1 to CQ3 are the prioritized questions (Priority 1 to 3), implemented in Assignment 8 and extended in Assignment 9 with inferred final feasibility judgments; CQ4 to CQ6 extend the same scenarios. All answers follow the shared evaluation rule in Section IV and use the axioms and rules in OE_9_TeamAffordance.rdf (rules S1 and S3) and the individuals and rules in OE_9_TeamAffordance_Individuals.rdf (rules S4 to S10). Assignment 11: the current files are OE_11_TeamAffordance.rdf and OE_11_TeamAffordance_Individuals.rdf, and the queries are in OE_11_TeamAffordance_Queries.txt. The test individuals now use the namespace https://tw.rpi.edu/ontology-engineering/oe2026/physical-properties/individuals/ (prefix ind: in the query file), and the rules carry the descriptive names listed in Section IV.

Prioritized competency questions (implemented and tested in Assignments 8 and 9)

Priority 1 — CQ1: Can a rigid plastic card cut a carrot, and why? (Scenario 1; depends on inference)

Priority 2 — CQ2: Which of a rigid plastic card, a rolling pin, and a kitchen sponge can cut the same carrot? (Scenario 2; depends on inference) - answered in SPARQL using OWL 2 DL + SWRL entailment (Assignment 9; see OE_9_TeamAffordance_SPARQLQueries.txt, Q2.1-Q2.4); Assignment 10: a fourth object, a metal spatula with unspecified edge thickness, was added so the answer shows all three Scenario 2 groups; Assignment 11: Q2.2 rewritten to read the inferred hasUnresolvedRequirement and Q2.3 simplified (no CONCAT), following instructor feedback

Priority 3 — CQ3: Can a knife and a pair of scissors both meet the CUT requirements, and how do their part structures differ? (Scenario 3; depends on inference)

Additional competency questions

Priority 4 — CQ4: Why is the card returned while the sponge is excluded? (Scenario 2; implemented in Assignment 9; depends on inference; SPARQL queries Q4.1 and Q4.2 added in Assignment 10 and merged into the single query Q4.1 in Assignment 11)

Priority 5 — CQ5: If the edge thickness of a candidate is unknown, what can be concluded and what information is missing? (Scenario 2 and alternate flow; implemented in Assignment 9; depends on inference; SPARQL query Q5.1 added in Assignment 11)

Priority 6 — CQ6: What feature levels must a candidate have to cut a high-hardness target? (Scenario 1; implemented as a classification)

Resource Note: EPIC-KITCHENS-100 provides the kitchen action vocabulary from which CUT was selected; the features, parts, and requirements used in the competency questions come from the CUT-specific annotations described in Section VIII.

CQ1 (Priority 1) — Can a rigid plastic card be used to cut a carrot, and why?

CQ1.1— Can a rigid plastic card be used to cut a carrot?

Sample Answer from Ontology

Yes. The ontology infers that the card satisfies both modeled CUT requirements and assigns the assessment the feasibility judgment Feasible.

Terms Used from Ontology

Assessment, WholeObject, Action, ActionRequirement, FeasibilityJudgment, TwoCUTRequirementsSatisfiedAssessment; hasCandidate, hasTarget, hasAction, satisfiesRequirement, hasJudgment; Feasible

Semantic Processes Involved

Individuals used: cardCarrotCutAssessment, cardCandidate, carrotTarget, cut, cutRigidityRequirement, cutEdgeThicknessRequirement, and feasibleJudgment.

Reasoning: Rule S1 infers satisfaction of the two CUT requirements. Rule S3 classifies cardCarrotCutAssessment as a TwoCUTRequirementsSatisfiedAssessment. Rule S4 assigns feasibleJudgment to the assessment. The inferred judgment is retrieved to answer whether the action is feasible.

Depends on inference: yes.

Usage Scenario Covered

Scenario 1 — Verifying a Substitute Tool Before Acting

Description + Ontology Usage

An Assessment links the candidate card, target carrot, and CUT action. The ontology evaluates whether the candidate satisfies the modeled action requirements and uses reasoning rules to infer a feasibility judgment. The query retrieves this judgment to determine whether the card can perform the action under the represented conditions.

CQ1.2 — Why can a rigid plastic card be used to cut a carrot?

Sample Answer from Ontology

The carrot has high hardness. For a high-hardness target, the CUT requirement mappings specify high rigidity and low edge thickness. The plastic card has high rigidity and low edge thickness, satisfying both requirements. Consequently, the assessment is classified as a TwoCUTRequirementsSatisfiedAssessment and receives the judgment Feasible.

Terms Used from Ontology

Assessment, WholeObject, Action, PhysicalFeature, FeatureValue, FeatureLevel, ActionRequirement, RigidityRequirement, ThicknessRequirement, RequirementMapping, HighHardnessRigidityMapping, TwoCUTRequirementsSatisfiedAssessment; hasCandidate, hasTarget, hasAction, hasRequirement, concernsFeature, hasFeatureValue, hasFeatureType, hasLevel, affectsRequirement, usesTargetFeatureType, usesTargetLevel, specifiesRequiredLevel, satisfiesRequirement, hasJudgment; Feasible, FeasibilityJudgment

Semantic Processes Involved

Individuals used: cardCarrotCutAssessment, cardCandidate, carrotTarget, cut, cardRigidityValue, cardEdgeThicknessValue, carrotHardnessValue, cutRigidityRequirement, cutEdgeThicknessRequirement, cutRigidityMapping, cutEdgeThicknessMapping, and feasibleJudgment. The test individuals testCandidateA and testAAssessment are also used to validate the reasoning.

Reasoning: Rule S1 matches the carrot's hardness level with the target conditions specified by each RequirementMapping and compares the card's feature levels with the required levels. The reasoner infers that cardCarrotCutAssessment satisfies cutRigidityRequirement and cutEdgeThicknessRequirement. Rule S3 classifies the assessment as a TwoCUTRequirementsSatisfiedAssessment, and rule S4 assigns feasibleJudgment. The query retrieves the target condition, requirement mappings, required feature levels, actual candidate feature levels, and inferred requirement satisfaction to explain the judgment. The test individuals receive the same inferences under equivalent feature conditions.

Depends on inference: yes.

Usage Scenario Covered

Scenario 1 — Verifying a Substitute Tool Before Acting

Description + Ontology Usage

An Assessment connects the candidate, target, and action. Each FeatureValue has exactly one feature type and one level, allowing the card's physical features to be compared with the requirements specified by the applicable RequirementMapping individuals. The ontology explains feasibility through the satisfaction of both modeled CUT requirements. Because both requirements are satisfied and neither is in conflict, rule S4 assigns Feasible according to the shared evaluation rule in Section IV.

CQ2 (Priority 2) — Given the same carrot, which of the following candidates satisfy the CUT requirements: a rigid plastic card, a rolling pin, a kitchen sponge, or a metal spatula (added in Assignment 10)?

Sample Answer from Ontology

Only the card. All three candidates are assessed against the same requirements: high rigidity and low edge thickness. The card has high rigidity and low edge thickness and satisfies both. The rolling pin satisfies the rigidity requirement, but its high edge thickness conflicts with the required low level. The sponge has low rigidity and high edge thickness and satisfies neither. Deformability is represented as low rigidity, so the sponge conflicts with the rigidity requirement rather than lacking information. With the reasoner running, the rules also infer the final judgment: hasJudgment feasibleJudgment for the card’s assessment, and hasJudgment infeasibleJudgment for the rolling pin and sponge assessments (Assignment 9). Assignment 10: the metal spatula has high rigidity, but its edge thickness was not supplied. It satisfies the rigidity requirement, has no conflict, and receives

insufficientInformationJudgment (rule S8), so the answer now contains all three groups of Scenario 2: qualifying (card), conflicting (rolling pin, sponge) and incomplete (spatula).

Terms Used from Ontology

Assessment, WholeObject, FeatureValue, FeatureLevel, ActionRequirement, RequirementMapping, TwoCUTRequirementsSatisfiedAssessment; hasCandidate, hasTarget, hasAction, hasFeatureValue, hasFeatureType, hasLevel, affectsRequirement, usesTargetFeatureType, usesTargetLevel, specifiesRequiredLevel, satisfiesRequirement; added in Assignment 9: FeasibilityJudgment, Feasible, Infeasible; conflictsWithRequirement, hasJudgment; added in Assignment 10: InsufficientInformation; hasUnspecifiedFeatureType; added in Assignment 11: hasUnresolvedRequirement

Semantic Processes Involved

Individuals used: cardCarrotCutAssessment, rollingPinCarrotCutAssessment, spongeCarrotCutAssessment, the three candidates and their feature values, and carrotTarget. Reasoning: rule S1 infers the satisfied requirements of each assessment independently (both requirements for the card, rigidity only for the rolling pin, and none for the sponge), and rule S3 classifies only cardCarrotCutAssessment as a TwoCUTRequirementsSatisfiedAssessment. A requirement that is not satisfied is reported as a conflict only when the candidate has a represented value of that feature at a different level. Depends on inference: yes. Assignment 9: rules S5 and S6 infer conflictsWithRequirement (edge thickness for the rolling pin; rigidity and edge thickness for the sponge), rule S10 classifies the card's assessment as a TwoCUTRequirementsSatisfiedAssessment, and rules S4 and S7 then assign feasibleJudgment (card) and infeasibleJudgment (rolling pin, sponge). Additional individuals used: feasibleJudgment, infeasibleJudgment. Assignment 10: new individuals spatulaCandidate, spatulaRigidityValue and spatulaCarrotCutAssessment, with spatulaCandidate hasUnspecifiedFeatureType edgeThickness; rule S1 infers that the spatula satisfies the rigidity requirement and rule MarkUnspecifiedFeatureRequirementAsUnresolved infers hasUnresolvedRequirement for the edge-thickness requirement and rule InsufficientWhenEdgeUnresolved (formerly S8) assigns insufficientInformationJudgment.

Usage Scenario Covered

Scenario 2 — Choosing a Substitute from Available Objects

Description + Ontology Usage

Each candidate has its own Assessment with the same target and action, so the same mappings apply to every candidate, and differences in the results can be traced to differences in feature levels.

SPARQL Implementation (new in Assignment 9)

CQ2 is answered by four queries in OE_9_TeamAffordance_SPARQLQueries.txt. Q2.1 (main) retrieves the inferred hasJudgment of each candidate's assessment. Q2.2 retrieves the inferred satisfied and conflicting requirements, which make up the explanation returned to the task planner. Both require OWL 2 DL entailment with the SWRL rules (Protégé Snap SPARQL Query plugin with HermiT). Without reasoning they return no rows, because these properties are never asserted. Q2.3 derives the same verdicts from the asserted feature values alone (standard SPARQL Query tab) as a cross-check. Q2.4 checks every assessment of the carrot, including the neutral-name test individuals, and returns all three groups of Scenario 2: feasible, infeasible and insufficient information. Expected and validated answer: card – feasible; rolling pin – infeasible (edge-thickness conflict); sponge – infeasible (rigidity

and edge-thickness conflicts). Assignment 10 update (OE_10_TeamAffordance_Queries.txt): the same four queries now include the spatula. Q2.1 returns 4 rows (card feasible; rolling pin and sponge infeasible; spatula insufficient information), Q2.2 adds the spatula's missing feature (edgeThickness), Q2.3 marks the spatula INSUFFICIENT INFORMATION from the asserted data alone, and Q2.4 returns 9 rows. All results were validated with HermiT on the merged ontology. Assignment 11 update (OE_11_TeamAffordance_Queries.txt), following instructor feedback: Q2.2 now reports satisfied, conflicting and unresolved requirements using only inferred properties of each assessment (satisfiesRequirement, conflictsWithRequirement and the new hasUnresolvedRequirement) and returns 8 rows. Q2.3 was rewritten without CONCAT, BIND or GROUP BY; for each object and feature it lists the level the carrot requires and the level the object has (8 rows, with an empty value for the spatula's edge thickness). Q2.1 (4 rows) and Q2.4 (9 rows) are unchanged apart from the new individuals namespace.

CQ4 (Priority 4) — Why is the rigid plastic card returned as a qualifying candidate for cutting the carrot while the kitchen sponge is excluded?

Sample Answer from Ontology: The two candidates differ in the features that the CUT requirements constrain. For the high-hardness carrot, CUT requires high rigidity and low edge thickness. The card has high rigidity and low edge thickness and satisfies both requirements. The sponge has low rigidity and high edge thickness; both values are represented and both differ from the required levels, so the sponge conflicts with both requirements.

Terms Used from Ontology

Assessment, FeatureValue, FeatureLevel, ActionRequirement, RequirementMapping; satisfiesRequirement, conflictsWithRequirement, concernsFeature, hasFeatureValue, hasLevel, specifiesRequiredLevel, hasJudgment; Feasible, Infeasible

Semantic Processes Involved

Individuals used: cardCarrotCutAssessment, spongeCarrotCutAssessment, cardRigidityValue, cardEdgeThicknessValue, spongeRigidityValue, spongeEdgeThicknessValue, cutRigidityMapping, and cutEdgeThicknessMapping. Reasoning: the explanation traces each requirement to its mapping, its required level, and the feature value of each candidate, using the satisfied requirements inferred for CQ2. Rules S5 and S6 infer that the sponge's assessment conflicts with both requirements, and rule S7 assigns infeasibleJudgment; the card's assessment satisfies both requirements and receives feasibleJudgment (rules S1, S10, and S4). Depends on inference: yes.

Usage Scenario Covered

Scenario 2 — Choosing a Substitute from Available Objects

Description + Ontology Usage

Supports the explanation step of Scenario 2, in which the planner must tell the user why an object was rejected. SPARQL Implementation (new in Assignment 10): Q4.1 lists, for the card and the sponge, each CUT requirement with its inferred status (satisfiesRequirement or conflictsWithRequirement), the level required by the applicable

mapping, the object's actual level, and the inferred judgment (4 rows). In Assignment 11 the former Q4.2 was merged into Q4.1, following instructor feedback: every card row is satisfied and every sponge row is in conflict on the same two requirements, so the single query also shows the deciding requirements, cutRigidityRequirement and cutEdgeThicknessRequirement. The query needs OWL 2 DL + SWRL entailment (Snap SPARQL with HermiT) and returns 0 rows without a reasoner.

CQ3 (Priority 3) — Can a knife and a pair of scissors both meet the CUT requirements for the same carrot, and how does the ontology represent the difference in their part structures?

Sample Answer from Ontology

Both tools satisfy the two modeled CUT requirements for the same high-hardness carrot: high rigidity and low edge thickness. Their assessments each receive feasibleJudgment. The knife is represented with one blade part, while the scissors are represented with two explicitly distinct blade parts that interact. The reasoner classifies scissorsCandidate as both MultiPartObject and InteractingPartObject. Neither classification is inferred for knifeCandidate. Under open-world semantics, this does not prove that the knife cannot belong to either class or that it has exactly one part.

Terms Used from Ontology

WholeObject, ObjectPart, MultiPartObject, InteractingPartObject, Assessment, FeatureValue, FeatureLevel, RequirementMapping; hasPart, isPartOf, interactsWith, hasFeatureValue, hasFeatureType, hasLevel, hasCandidate, hasTarget, hasAction

Semantic Processes Involved

Individuals used: knifeCandidate, knifeBlade, scissorsCandidate, scissorsBlade1, scissorsBlade2 and their feature values; knifeCarrotCutAssessment, scissorsCarrotCutAssessment, carrotTarget, cut, cutRigidityRequirement, cutEdgeThicknessRequirement, cutRigidityMapping, cutEdgeThicknessMapping, and feasibleJudgment. Structural regression controls use testCandidate3 and testCandidate4, each with two explicitly distinct ObjectPart individuals.

The distinct scissors blades support the inference that scissorsCandidate is a MultiPartObject. Their interaction also supports classification as an InteractingPartObject. Both scissors interaction directions are already asserted. For testCandidate4, only testCandidate4Part1 interactsWith testCandidate4Part2 is asserted; symmetry infers the reverse relationship. The inverse of hasPart also infers isPartOf relationships, including knifeBlade isPartOf knifeCandidate.

testCandidate4 receives both structural classifications. testCandidate3 is inferred to be a MultiPartObject, but no interaction or InteractingPartObject membership is inferred for it. Missing interaction assertions do not prove noninteraction. These two candidates test structural classification only; they have no CUT assessments or feature values. Depends on inference: yes.

Usage Scenario Covered

Scenario 3 — Comparing Alternative Part Structures for the Same Action

Description + Ontology Usage

Blade features support requirement satisfaction through hasPart and rule S9. Rule S10 combines satisfaction of the two CUT requirements at assessment level, and S4 assigns feasibleJudgment. Different parts may supply different requirements; neither every part nor the same part must satisfy both requirements. Both scissors blades happen to carry both required features in the current data. Interaction supports structural classification but is not currently a CUT prerequisite. Inferring conflicts from part features remains an open issue.

SPARQL Implementation — Assignment 9

CQ3 is evaluated through queries Q3.1–Q3.7 in OE_9_TeamAffordance_SPARQLQueries.txt. The queries check represented parts, explicit distinctness, interactions, structural classifications, inverse part relationships, CUT judgments, and supporting blade features.

All seven queries returned the expected results in Protégé 5.6.9 using the standard SPARQL Query view for asserted data and Snap SPARQL 6.0.0 with HermiT for inferred results. Q3.4 returned five structural-classification rows: two each for scissorsCandidate and testCandidate4, and one for testCandidate3. Q3.6 returned four rows showing that the knife and scissors assessments each satisfy both CUT requirements and receive feasibleJudgment. Screenshots of these results are included in Report Section 2.3.

CQ5 (Priority 5) — If the rigidity of a candidate is known but its edge thickness is not represented, can the system decide whether it can cut the carrot, and what information is missing?

Sample Answer from Ontology

No final decision can be made. testCandidateC has high rigidity, so it satisfies the rigidity requirement, but its edge thickness is not represented. Under open-world semantics this is not evidence of infeasibility; the result is insufficient information, and the missing information is the edge thickness of the candidate.

Terms Used from Ontology

Assessment, FeatureValue, ActionRequirement, FeasibilityJudgment, InsufficientInformation; satisfiesRequirement, concernsFeature, hasFeatureValue, hasFeatureType, hasUnspecifiedFeatureType, hasJudgment, hasUnresolvedRequirement

Semantic Processes Involved

Individuals used: testCandidateC, testCRigidityValue, testCAssessment, spatulaCandidate, spatulaRigidityValue, spatulaCarrotCutAssessment (added in Assignment 10), carrotTarget, and cutEdgeThicknessRequirement. Reasoning: the reasoner infers that the assessment satisfies the rigidity requirement and does not classify it as a TwoCUTRequirementsSatisfiedAssessment; the missing feature is found by identifying the requirement whose concerned feature (edgeThickness) has no value for the candidate; because that feature is recorded as unspecified (hasUnspecifiedFeatureType edgeThickness), rule MarkUnspecifiedFeatureRequirementAsUnresolved infers hasUnresolvedRequirement for the edge-thickness requirement and rule InsufficientWhenEdgeUnresolved (formerly S8) assigns insufficientInformationJudgment. Depends on inference: yes.

Usage Scenario Covered

Scenario 2 — Choosing a Substitute from Available Objects (Alternate Flow)

Description + Ontology Usage

Tests the requirement that missing information is distinguished from conflicting information. SPARQL Implementation (new in Assignment 11): Q5.1 (Snap SPARQL with HermiT) returns every carrot assessment that has an inferred unresolved requirement, together with its judgment, the requirement it still satisfies and the missing feature. Expected answer, validated with HermiT on the merged OE_11 files: 2 rows, spatulaCandidate and testCandidateC, each with insufficientInformationJudgment, cutRigidityRequirement satisfied, cutEdgeThicknessRequirement unresolved and edgeThickness missing; 0 rows without a reasoner.

CQ6 (Priority 6) — What feature levels must a candidate have to cut a target with high hardness?

Sample Answer from Ontology

High rigidity and low edge thickness. When the hardness of the target is high, the mapping for the CUT rigidity requirement specifies a high level, and the mapping for the edge-thickness requirement specifies a low level.

Terms Used from Ontology

Action, ActionRequirement, RequirementMapping, HighHardnessRigidityMapping, FeatureLevel; hasRequirement, affectsRequirement, usesTargetFeatureType, usesTargetLevel, specifiesRequiredLevel, concernsFeature

Semantic Processes Involved

Individuals used: cut, cutRigidityRequirement, cutEdgeThicknessRequirement, cutRigidityMapping, cutEdgeThicknessMapping, hardness, high, and low. Reasoning: the system retrieves the mappings for the requirements of cut whose target condition is hardness = high, and the reasoner classifies cutRigidityMapping as a HighHardnessRigidityMapping from its property values. Depends on inference: yes (classification).

Usage Scenario Covered

Scenario 1 — Verifying a Substitute Tool Before Acting

Description + Ontology Usage

Uses only the reusable individuals in the main ontology. This is the first step of every assessment: determining what the target demands before any candidate is checked.

VIII. Resources

Data Availability
EPIC-KITCHENS-100 is publicly available and provides the source kitchen-domain action vocabulary from which Cut was selected for the initial ontology scope. However, EPIC-KITCHENS does not directly provide the physical

properties, object-part descriptions, or action-requirement relations needed for the proposed feasibility reasoning. These are supplied by a small curated subset of feasibility and counterfactual annotations generated as part of ongoing affordance-reasoning research. For the course project, only data relevant to Cut and the competency questions defined in Section VII will be used.

Knowledge Bases, Repositories, or other Data Sources

Data	Type	Characteristics	Description	Owner	Source	Access Policies & Usage
EPIC-KITCHENS-100	Remote dataset / repository	Egocentric kitchen videos with annotated manipulation actions and object interactions	Provides the source kitchen-domain action vocabulary and human-object interaction context from which Cut is selected for the initial ontology scope. It does not directly provide the physical-property or action-requirement knowledge used for feasibility reasoning.	EPIC-KITCHENS / University of Bristol and collaborators	EPIC-KITCHENS dataset website / annotation repository Link (full dataset): https://epic-kitchens.github.io/ Link (annotation dataset): https://github.com/epic-kitchens/epic-kitchens-100-annotations	CC BY-NC 4.0. Non-commercial use is permitted with appropriate attribution, a link to the license, and an indication of any changes. The dataset is used to establish the source action vocabulary and is not queried directly during ontology reasoning. License: https://github.com/epic-kitchens/epic-kitchens-100-annotations/blob/master/license.txt
Derived Feasibility Scenario Annotations	Local / experimentally generated dataset	A curated set of feasible and infeasible examples for the action Cut, including the relevant object, object part, physical properties, target context, and feasibility	Provides the instance-level evidence used to populate the course ontology with examples of objects, object parts, relevant physical properties, and feasible or	Hyejin Park / LACAI Lab	Internal project resource; no public URL available at this stage.	Internal research resource; not publicly available. A curated CUT-specific subset will be shared with the course team for ontology development

		outcome. The resource provides instance-level evidence used to populate and test the course ontology and to support the competency questions in Section VII.	infeasible outcomes. Only the Cut-specific subset will be used for this course project.			and evaluation. Redistribution outside the project is not authorized at this stage. No public license has been assigned.
Derived Counterfactual Contrast Annotations	Local / experimentally generated dataset	Paired examples in which changes to an object property or relevant part are associated with changes in action feasibility	Paired or contrastive examples in which a change to a relevant object part or physical property is associated with a change in cutting feasibility. The resource is used as supporting evidence when identifying or refining which properties are relevant to the ActionRequirements for Cut	Hyejin Park / LACAI Lab	Generated through ongoing affordance-reasoning experiments	Internal research resource; not publicly available. A limited CUT-specific subset may be shared with the course team as supporting evidence for identifying and refining property–requirement relationships. No public license has been assigned. Any further sharing is subject to the research project's data-use policy.

External Ontologies, Vocabularies, or other Model Services

Resource	Language	Description	Owner	Source	Describes/Uses	Access Policies & Usage
Word Net	English	Supports terminology normalization and sense distinction for action and object terms used in the	Princeton University	Princeton WordNet Link: https://wordnet.princeton.edu/	Action and object terminology derived from EPIC-	Princeton WordNet License. Use, modification, and

		ontology. It is not used to determine physical feasibility.			KITCHENS and the experimental annotations	distribution are permitted without a fee, subject to retaining the required copyright notice and license statements. Used as an optional terminology resource, not for determining physical feasibility. License: https://wordnet.princeton.edu/license-and-commercial-use
Relations Ontology (RO)	OWL / OBO	Provides standardized relation types intended for reuse across ontologies. It will be reviewed to determine whether general relations needed in this project, such as part-whole relations, can reuse existing semantics before new project-specific relations are introduced.	OBO Relations Ontology community / OBO Foundry	Official Website Link: https://obofoundry.org/ontology/ro.html	Relations among ontology entities, particularly reusable general-purpose relations such as part-whole relations.	CC0 1.0 Universal (Public Domain Dedication). RO is available for reuse and adaptation. The original ontology and term identifiers should be retained when reusing existing relations, and the source should be cited. Used as an optional reference for

						relation reuse and ontology-design decisions. License: https://github.com/oborel/obo-relations/blob/master/LICENSE.txt

Other Resources, Service, or Triggers *(e.g., event notification services, application services, etc.)*

Resource	Type	Description	Owner	Source	Access Policies & Usage
(sensor or external service name)		*Include a description of the resource as well as availability, if applicable*	*Primary owner of the service*	***Application or service URL**; if subscription based, include subscription and any subscription owner*	

IX. References and Bibliography

-

[1] EPIC-KITCHENS, "EPIC-KITCHENS dataset." Accessed: Oct. 14, 2026. [Online]. Available: https://epic-kitchens.github.io/

[2] EPIC-KITCHENS, "EPIC-KITCHENS-100 annotations," GitHub repository. Accessed: Oct. 14, 2026. [Online]. Available: https://github.com/epic-kitchens/epic-kitchens-100-annotations

[3] Princeton University, "WordNet." Accessed: Oct. 14, 2026. [Online]. Available: https://wordnet.princeton.edu/

[4] OBO Foundry, "Relations Ontology." Accessed: Oct. 14, 2026. [Online]. Available: https://obofoundry.org/ontology/ro.html

[5] Object Management Group, “OMG Unified Modeling Language (OMG UML), version 2.5.1,” Dec. 2017. Accessed: Oct. 14, 2026. [Online]. Available: https://www.omg.org/spec/UML/2.5.1/

[6] A. Chemero, “An outline of a theory of affordances,” Ecological Psychology, vol. 15, no. 2, pp. 181–195, 2003, doi: 10.1207/S15326969ECO1502_5.

[7] Y. Jiang, M. Yang, Z. Li, and J. Pujara, “AFFORDANCE20Q: Evaluating affordance reasoning from physical properties,” arXiv:2606.14240v2, 2026. [Online]. Available: https://arxiv.org/abs/2606.14240v2

[8] H. Wang et al., “Affordance-R1: Reinforcement learning for generalizable affordance reasoning in multimodal large language models,” Proceedings of the AAAI Conference on Artificial Intelligence, vol. 40, no. 12, pp. 9738–9746, 2026, doi: 10.1609/aaai.v40i12.37937.

[9] C. Xie et al., “RelationLMM: Large multimodal model as open and versatile visual relationship generalist,” IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 47, no. 5, pp. 3515–3529, May 2025, doi: 10.1109/TPAMI.2025.3531452.

[10] B. Barati and E. Karana, “Affordances as materials potential: What design can do for materials development,” International Journal of Design, vol. 13, no. 3, pp. 105–123, 2019.

[11] M. Qu et al., “RIO: A benchmark for reasoning intention-oriented objects in open environments,” in Advances in Neural Information Processing Systems, vol. 36, 2023. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/hash/8644353f7d307baaf29bc1e56fe8e0ec-Abstract-Datasets_and_Benchmarks.html

[12] Z. Wang, Y. Zhong, J. Fu, H. I. Christensen, and J. Gao, “AFUN: Towards an affordance foundation model for functionality understanding,” arXiv:2606.02551, 2026. [Online]. Available: https://arxiv.org/abs/2606.02551

[13] J. McCusker et al., “ARCLIGHT: Automated clustering and curriculum learning guided by human training,” in Proceedings of the Special Session on Harmonising Generative AI and Semantic Web Technologies (HGAIS 2024), CEUR Workshop Proceedings, vol. 3953, 2025. [Online]. Available: https://ceur-ws.org/Vol-3953/366.pdf

[14] A. Myers, C. L. Teo, C. Fermüller, and Y. Aloimonos, “Affordance detection of tool parts from geometric features,” in Proceedings of the IEEE International Conference on Robotics and Automation (ICRA), Seattle, WA, USA, 2015, pp. 1374–1381, doi: 10.1109/ICRA.2015.7139369.

X. Notes

There is always some piece of information that is required that has no other place to go. This is the place for that information.